

Lab3实验报告

思考题

Thinking 3.1

Thinking 3.1 请结合 MOS 中的页目录自映射应用解释代码中 `e->env_pgdir[PDX(UVPT)] = PADDR(e->env_pgdir) | PTE_V` 的含义。

- 这行代码的作用是将 `e` 的页目录的物理地址写入到其自身的页目录中的特定位置（由 `PDX(UVPT)` 计算得到），并标记这个页目录项为有效。这样，当用户程序通过 `UVPT` 这个虚拟地址访问时，实际上会访问到其自身的页目录。

Thinking 3.2

Thinking 3.2 `elf_load_seg` 以函数指针的形式，接受外部自定义的回调函数 `map_page`。请你找到与之相关的 `data` 这一参数在此处的来源，并思考它的作用。没有这个参数可不可以？为什么？

`elf_load_seg` 在 `lib/elfloader.c` 中实现：

```
1  int elf_load_seg(Elf32_Phdr *ph, const void *bin, elf_mapper_t map_page,
2  void *data) {
3      u_long va = ph->p_vaddr;
4      size_t bin_size = ph->p_filesz;
5      size_t sgsize = ph->p_memsz;
6      u_int perm = PTE_V;
7      if (ph->p_flags & PF_W) {
8          perm |= PTE_D;
9      }
10     int r;
11     size_t i;
12     u_long offset = va - ROUNDDOWN(va, PAGE_SIZE);
13     if (offset != 0) {
14         if ((r = map_page(data, va, offset, perm, bin,
15             MIN(bin_size, PAGE_SIZE - offset))) != 0) {
16             return r;
17         }
18     }
19     /* Step 1: load all content of bin into memory. */
20     for (i = offset ? MIN(bin_size, PAGE_SIZE - offset) : 0; i < bin_size;
21         i += PAGE_SIZE) {
22         if ((r = map_page(data, va + i, 0, perm, bin + i, MIN(bin_size - i,
23             PAGE_SIZE))) !=
24             0) {
25             return r;
26         }
27     }
28     /* Step 2: alloc pages to reach `sgsize` when `bin_size` < `sgsize`. */
29     while (i < sgsize) {
30         if ((r = map_page(data, va + i, 0, perm, NULL, MIN(sgsize - i,
31             PAGE_SIZE))) != 0) {
32             return r;
33         }
34     }
```

```

30     i += PAGE_SIZE;
31 }
32 return 0;
33 }

```

指导书中提到：

- 最后两个参数用于接受一个自定义的回调函数 `map_page`，以及需要传递给回调函数的额外参数 `data`。每当 `elf_load_seg` 函数解析到一个需要加载到内存中的页面，会将有关的信息作为参数传递给回调函数，并由它完成单个页面的加载过程，而这里的 `load_icode_mapper` 就是 `map_page` 的具体实现。

`map_page` 的函数指针在 `include/elf.h` 中声明：

```

1 typedef int (*elf_mapper_t)(void *data, u_long va, size_t offset, u_int
2 perm, const void *src,
   size_t len);

```

其实现在 `kern/env.c` 的 `load_icode_mapper` 函数：

```

1 static int load_icode_mapper(void *data, u_long va, size_t offset, u_int
   perm, const void *src,
2                               size_t len) {
3     struct Env *env = (struct Env *)data;
4     struct Page *p;
5     int r;
6     /* Step 1: Allocate a page with 'page_alloc'. */
7     if ((r = page_alloc(&p)) != 0) {
8         return r;
9     }
10    /* Step 2: If 'src' is not NULL, copy the 'len' bytes started at 'src'
       into 'offset' at this
11        * page. */
12    if (src != NULL) {
13        memcpy((void *)page2kva(p) + offset, src, len);
14    }
15    /* Step 3: Insert 'p' into 'env->env_pgdir' at 'va' with 'perm'. */
16    return page_insert(env->env_pgdir, env->env_asid, p, va, perm);
17 }

```

综合以上代码，我们可以了解 `data` 参数

`data` 参数的作用：

- 环境上下文： `data` 参数允许 `elf_load_seg` 函数将 `env` 的上下文信息传递给 `map_page` 回调函数。在 `load_icode_mapper` 的实现中， `data` 被强制转换为 `struct Env *` 类型，这样回调函数就可以访问和修改状态。
- 解耦： `data` 参数帮助解耦 `elf_load_seg` 和 `map_page` 的实现。 `elf_load_seg` 不需要知道 `map_page` 如何使用 `data`，只需要确保传递正确的参数。

没有 `data` 参数会带来以下问题：

- 限制灵活性：如果没有 `data` 参数， `map_page` 回调函数将无法直接接收额外的上下文信息，这会限制函数的用途，使其只能用于特定的场景。
- 增加复杂性：如果没有 `data` 参数，可能需要使用全局变量或其他间接方式来传递 `env` 的上下文信息，这会增加代码的复杂性并可能引入错误。

综上所述，`data` 参数在 `elf_load_seg` 和 `map_page` 之间起到了重要的桥梁作用，它允许传递环境的上下文信息，增加了代码的灵活性、可读性和可维护性。没有这个参数会带来不必要的复杂性和限制。

Thinking 3.3

Thinking 3.3 结合 `elf_load_seg` 的参数和实现，考虑该函数需要处理哪些页面加载的情况。

该函数需要处理以下几种页面加载的情况：

1. 段对齐情况：
 - 当段的虚拟地址 (`va`) 恰好与页面边界对齐时，即 `va` 是 `PAGESIZE` 的倍数，可以直接开始加载段内容到页面中。
2. 段非对齐情况：
 - 当段的虚拟地址 (`va`) 不是页面边界对齐时，需要先处理第一个不完整的页面。这涉及到将段的一部分内容加载到页面的偏移位置，并确保不会覆盖页面的其他部分。
3. 段内容小于页面大小：
 - 如果段的内容大小 (`bin_size`) 小于一个页面大小，需要将段内容加载到页面的开始位置，并可能需要将页面的剩余部分清零或保留原内容，具体取决于段的内存大小 (`sgsize`)。
4. 段内容跨越多个页面：
 - 当段的内容大小超过一个页面时，需要连续加载多个页面，直到整个段的内容都被加载完毕。
5. 段内存大小大于文件大小：
 - 如果段的内存大小 (`sgsize`) 大于文件大小 (`bin_size`)，说明段在内存中需要占用更多的空间，但文件中并没有相应的内容。这种情况下，需要分配足够的页面来满足内存大小，并将文件内容加载到这些页面的开始部分，剩余的部分可能需要清零。
6. 权限设置：
 - 根据段头部中的标志 (`pflags`)，设置页面的权限 (`perm`)。例如，如果段是可写的，权限应该包括写权限。
7. 错误处理：
 - 在页面分配、数据复制或页表插入过程中，如果发生错误（如内存不足），需要正确处理这些错误并返回相应的错误码。
8. 零填充：
 - 如果段的内存大小 (`sgsize`) 大于文件大小 (`bin_size`)，超出部分需要为零填充。
9. 页表更新：
 - 对于每个新分配的页面，需要将其插入到环境的页表中，以建立虚拟地址到物理地址的映射。

Thinking 3.4

- 这里的 `env_tf.cp0_epc` 字段指示了进程恢复运行时 PC 应恢复到的位置。我们要运行的进程的代码段预先被载入到了内存中，且程序入口为 `e_entry`，当我们运行进程时，CPU 将从 PC 所指的位置开始执行二进制码。

Thinking 3.4 思考上面这一段话，并根据自己在 Lab2 中的理解，回答：你认为这里的 `env_tf.cp0_epc` 存储的是物理地址还是虚拟地址？

- 是虚拟地址。操作系统通常为每个进程提供独立的虚拟地址空间，以实现进程隔离和保护。进程的代码段被加载到这个虚拟地址空间中，而不是直接加载到物理内存的特定位置。
- `e_entry` 是程序入口的虚拟地址，不是物理地址。当进程开始执行时，CPU 应该从虚拟地址 `e_entry` 开始执行，而不是某个物理地址。

- 当异常发生时，CPU 需要保存当前执行的虚拟地址，以便异常处理完成后能够恢复执行。如果保存的是物理地址，那么在异常处理过程中可能会因为地址空间的变化（如换页）而导致无法正确恢复。
- 在实际执行时，CPU 会通过页表将虚拟地址转换为物理地址。这个过程对程序员是透明的，程序员只需要关心虚拟地址。

Thinking 3.5

- 0 号异常的处理函数为 `handle_int`，表示中断，由时钟中断、控制台中断等中断造成
- 1 号异常的处理函数为 `handle_mod`，表示存储异常，进行存储操作时该页被标记为只读
- 2 号异常的处理函数为 `handle_tlb`，表示 TLB load 异常
- 3 号异常的处理函数为 `handle_tlb`，表示 TLB store 异常
- 8 号异常的处理函数为 `handle_sys`，表示系统调用，用户进程通过执行 `syscall` 指令陷入内核

Thinking 3.5 试找出 0、1、2、3 号异常处理函数的具体实现位置。8 号异常（系统调用）涉及的 `do_syscall()` 函数将在 Lab4 中实现。

- `handle_int` 在 `kern/genex.S` 实现：

```

1  NESTED(handle_int, TF_SIZE, zero)
2      mfc0    t0, CP0_CAUSE
3      mfc0    t2, CP0_STATUS
4      and     t0, t2
5      andi    t1, t0, STATUS_IM7
6      bnez    t1, timer_irq
7  timer_irq:
8      li      a0, 0
9      j       schedule
10  END(handle_int)

```

- `handle_mod` 和 `handle_tlb`，是通过 `genex.S` 文件中的宏函数 `BUILD_HANDLER` 定义的：

```

1  BUILD_HANDLER tlb do_tlb_refill
2  #if !defined(LAB) || LAB >= 4
3  BUILD_HANDLER mod do_tlb_mod
4  BUILD_HANDLER sys do_syscall
5  #endif

```

通过宏函数实现：

```

1  .macro BUILD_HANDLER exception handler
2  NESTED(handle_\exception, TF_SIZE + 8, zero)
3      move    a0, sp
4      addiu   sp, sp, -8
5      jal     \handler
6      addiu   sp, sp, 8
7      j       ret_from_exception
8  END(handle_\exception)
9  .endm

```

Thinking 3.6

Thinking 3.6 阅读 `entry.S`、`genex.S` 和 `env_asm.S` 这几个文件，并尝试说出时钟中断 在哪些时候开启，在哪些时候关闭。

```

1  /*entry.S-----
   ----*/
2  .section .text.tlb_miss_entry
3  tlb_miss_entry:
4      j      exc_gen_entry
5
6  .section .text.exc_gen_entry
7  exc_gen_entry:
8      SAVE_ALL
9      mfc0    t0, CP0_STATUS
10     and     t0, t0, ~(STATUS_UM | STATUS_EXL | STATUS_IE)
11     mtc0    t0, CP0_STATUS
12     mfc0    t0, CP0_CAUSE
13     andi    t0, 0x7c
14     lw      t0, exception_handlers(t0)
15     jr      t0
16
17  /*genex.S-----
   ----*/
18  .macro BUILD_HANDLER exception handler
19  NESTED(handle_\exception, TF_SIZE + 8, zero)
20      move    a0, sp
21      addiu   sp, sp, -8
22      jal     \handler
23      addiu   sp, sp, 8
24      j      ret_from_exception
25  END(handle_\exception)
26  .endm
27
28  .text
29
30  FEXPORT(ret_from_exception)
31      RESTORE_ALL
32      eret
33
34  NESTED(handle_int, TF_SIZE, zero)
35      mfc0    t0, CP0_CAUSE
36      mfc0    t2, CP0_STATUS
37      and     t0, t2
38      andi    t1, t0, STATUS_IM7
39      bnez    t1, timer_irq
40  timer_irq:
41      li      a0, 0
42      j      schedule
43  END(handle_int)
44
45  BUILD_HANDLER tlb do_tlb_refill
46
47  #if !defined(LAB) || LAB >= 4
48  BUILD_HANDLER mod do_tlb_mod
49  BUILD_HANDLER sys do_syscall
50  #endif
51
52  BUILD_HANDLER reserved do_reserved
53
54  /*env_asm.S-----
   -----*/

```

```

55 .text
56 LEAF(env_pop_tf)
57 .set reorder
58 .set at
59     mtc0    a1, CP0_ENTRYHI
60     move    sp, a0
61     RESET_KCLOCK
62     j       ret_from_exception
63 END(env_pop_tf)

```

- 时钟中断开启的情况：

1. 系统初始化时：

- 在系统启动过程中，时钟中断可能会被开启以允许时钟中断处理。

2. 从异常处理返回时：

- 在 `ret_from_exception` 标签处，`RESTORE_ALL` 宏会恢复 `CP0_STATUS` 寄存器的值，如果之前允许中断，那么中断将会被重新开启。
- `eret` 指令执行时，会从异常处理返回到之前的执行状态，如果之前的执行状态允许中断，那么中断将会被开启。

3. 进程切换时：

- 在 `env_pop_tf` 函数中，`RESET_KCLOCK` 宏会开启时钟中断，以便新进程可以接收时钟中断。

- 时钟中断关闭的情况：

1. 进入异常处理时：

- 在 `exc_gen_entry` 标签处，通过以下代码关闭了中断：

```

1  mfc0    t0, CP0_STATUS
2  and     t0, t0, ~(STATUS_UM | STATUS_EXL | STATUS_IE)
3  mtc0    t0, CP0_STATUS

```

- 这段代码将 `CP0_STATUS` 寄存器的 `IE` 位清零，从而关闭了时钟中断。

2. 处理特定异常时：

- 在 `handle_int` 函数中，处理时钟中断之前，中断已经被关闭，因为进入异常处理时就已经关闭了中断。

Thinking 3.7

Thinking 3.7 阅读相关代码，思考操作系统是怎么根据时钟中断切换进程的。

- 时钟中断触发进程切换的流程

- 时钟硬件产生中断，CPU响应并跳转到中断处理函数。
- 中断处理函数会执行必要的处理，然后可能会设置一个标志或直接调用 `schedule` 函数来触发进程切换。
- `schedule` 函数被调用，通常在时钟中断处理结束后或当前进程主动放弃CPU时（通过设置 `yield` 参数）。

- `schedule` 函数执行流程

- `count` 变量减1，表示当前进程的时间片减少。
- 如果满足以下任一条件，则需要从 `env_sched_list` 中选择一个新的进程来运行
 - `yield` 参数被设置（非零），表示当前进程主动放弃CPU。
 - `count` 减少到0，表示当前进程的时间片用完。

- `curenv`（当前进程）为 `NULL`，表示没有当前进程。
- `curenv` 的状态不是 `ENV_RUNNABLE`，表示当前进程不可运行。
- 如果当前进程存在且是可运行的，将其从 `env_sched_list` 中移除，并插入到列表的尾部，以便其他进程有机会被调度。
- 从 `env_sched_list` 的头部选择一个新的进程 `e`。
- 如果列表为空，表示没有可运行的进程，系统会调用 `panic` 函数。
- 将 `count` 设置为新进程的优先级（`env_pri`）。
- 调用 `env_run(e)` 函数，该函数会保存当前进程的状态，加载新进程的状态，并开始执行新进程。

难点与体会

lab3的实验对我来说相当困难，但仍然比lab2好理解得多，主要是因为对于一些数据结构：页、链表等等有所理解了。

- 进程初始化：主要涉及对Env结构体的理解，基本思路和lab2的页面初始化类似。
- 进程创建：使用了初始化钟创建的模板页表，需要一定理解，实际上就是利用内存拷贝减少物理空间开销。二进制程序镜像对我而言也非常抽象，后来才知道就是一段数据。
- 进程调度：涉及大量计组知识，难点在于重新拾起mips的中断策略。
- 其他难点：
 - 第一次接触函数指针时比较难以理解。
 - 由于计组知识不牢，对有关mips的cp0的内容一知半解，极大地影响了指导书阅读。